

PHP UNIT-4 (ChatGpt)

2 MARKS QUESTION

1. What is the role of HTML form in web development.

HTML forms allow users to input and submit data to a server. They are essential for user interactions like login, registration, and search, enabling communication between front-end and back-end.

2. What is Use of Action Attribute in HTML Form with Example

The action attribute specifies where to send the form data after submission.

Example:

html

Copy Edit

```
<form action="submit.php" method="post">
  <input type="text" name="name">
  <input type="submit">
</form>
```

3. Two Methods for Handling Form Data in PHP

- **\$_GET** – Retrieves data sent via GET method.
- **\$_POST** – Retrieves data sent via POST method.

4. POST vs GET Methods in HTML Forms

Feature	POST	GET
Data Visible in URL	No	Yes
Data Limit	Large	2048 characters
Security	More secure	Less secure
Use Case	Login, forms	Search, links

5. Limitations of Using GET Method

- Data is visible in URL (less secure).
 - Limited data length (~2048 characters).
 - Cannot send binary data like files.
-

6. Security Considerations in PHP Form Handling

- Sanitize input (htmlspecialchars()).
 - Validate user data.
 - Use prepared statements to prevent SQL injection.
 - Avoid trusting user input directly.
-

7. Purpose of \$_SERVER['REQUEST_METHOD']

It checks the HTTP request method (GET, POST, etc.) used to access the page, allowing conditional form handling.

Example:

```
php Copy Edit  
  
if ($_SERVER['REQUEST_METHOD'] == 'POST') {  
    // handle form data  
}
```

8. What is MySQL and Its Use in Web Development

MySQL is an open-source relational database system used to store and manage data. In web development, it's used to handle dynamic data like user accounts, products, and content.

9. Four Data Types in MySQL

- INT – Integer numbers
 - VARCHAR – Variable-length text
 - DATE – Calendar date
 - BOOLEAN – True/False values
-

10. CHAR vs VARCHAR

Feature	CHAR	VARCHAR
Fixed Length	Yes	No
Storage	Always fixed	Variable
Padding	Padded with spaces	Not padded

11. Purpose of Auto-Increment Field with Example

Auto-increment automatically increases the value of a field (usually ID).

Example:

```
sql Copy Edit

CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50)
);
```

12. What is the MySQL Client

The MySQL client is a command-line interface used to interact with MySQL servers. It allows running queries, managing databases, and viewing data directly.

13. MySQL Client vs phpMyAdmin

Feature	MySQL Client	phpMyAdmin
Interface	CLI	Web GUI
Learning Curve	Higher	User-friendly
Installation	Lightweight	Browser-based

14. Four Common MySQL Commands

- CREATE DATABASE dbname;
- CREATE TABLE tablename (...);
- SELECT * FROM tablename;

- INSERT INTO tablename VALUES (...);

15. Establishing Connection to MySQL in PHP

```
php                                                                    Copy Edit

$conn = new mysqli("localhost", "root", "", "school");
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
```

16. Purpose of mysqli_query() with Example

Executes SQL queries in PHP.

Example:

```
php                                                                    Copy Edit

$sql = "SELECT * FROM students";
$result = mysqli_query($conn, $sql);
```

17. mysqli_num_rows() vs mysqli_affected_rows()

Function	Use
<code>mysqli_num_rows()</code>	Counts rows in SELECT result
<code>mysqli_affected_rows()</code>	Counts affected rows in INSERT, UPDATE, DELETE

18. Executing Simple MySQL Queries in PHP

```
php                                                                    Copy Edit

$sql = "SELECT * FROM students";
$result = mysqli_query($conn, $sql);
while ($row = mysqli_fetch_assoc($result)) {
    echo $row['name'];
}
```

19. Function to Retrieve Query Results in PHP

`mysqli_fetch_assoc()` is used to retrieve rows from a result set as associative arrays.

20. Count Number of Records Returned by a MySQL Query

```
php                                                                    Copy Edit

$result = mysqli_query($conn, "SELECT * FROM students");
$count = mysqli_num_rows($result);
echo "Total records: $count";
```

21. Updating Records in MySQL Using PHP

```
php                                                                    Copy Edit

$sql = "UPDATE students SET grade='A' WHERE id=1";
if (mysqli_query($conn, $sql)) {
    echo "Record updated";
}
```

22. PHP Code for Validating Email

```
php                                                                    Copy Edit

$email = "test@example.com";
if (filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "Valid email";
} else {
    echo "Invalid email";
}
```

23. `mysqli_fetch_array()` Constants

- `MYSQLI_ASSOC` – Returns associative array
- `MYSQLI_NUM` – Returns numeric array
- `MYSQLI_BOTH` – Returns both

4-6 MARKS QUESTION

1. Explain the role of HTML forms in web development and discuss two methods for handling form data in PHP. Provide examples to illustrate their usage.

1. Role of HTML Forms in Web Development

HTML forms are essential components of web development that allow users to input data and interact with a website. They are used for a variety of purposes such as:

- User authentication (login/signup)
- Feedback submission
- File uploads
- Online purchases
- Data collection (e.g., surveys, contact forms)

HTML forms provide an interface with input elements like:

- `<input>` (text, password, checkbox, radio)
- `<textarea>`
- `<select>`
- `<button>`

Once the user fills out the form and submits it, the data is sent to a server (typically using **HTTP POST or GET**) for processing.

2. Methods for Handling Form Data in PHP

PHP provides two primary global arrays to handle data submitted through forms:

- `$_GET`
- `$_POST`

Method 1: Handling form data using `$_GET`

`$_GET` retrieves form data sent via the URL (query string). It's visible in the browser's address bar and is best used for simple, non-sensitive data.

Example (GET Method):

HTML Form:

html

Copy Edit

```
<form action="get_example.php" method="get">
  Name: <input type="text" name="username"><br>
  <input type="submit" value="Submit">
</form>
```

PHP Script (get_example.php):

php

Copy Edit

```
<?php
$name = $_GET['username'];
echo "Hello, " . htmlspecialchars($name);
?>
```

Output: If you entered "John", the URL will be:

get_example.php?username=John

Output will be: Hello, John

Method 2: Handling form data using \$_POST

\$_POST retrieves form data sent through the HTTP request body. It's not visible in the URL and is suitable for sensitive or large data (e.g., passwords, text areas).

Example (POST Method):

HTML Form:

html

Copy Edit

```
<form action="post_example.php" method="post">
  Email: <input type="email" name="email"><br>
  Password: <input type="password" name="password"><br>
  <input type="submit" value="Login">
</form>
```

PHP Script (post_example.php):

php

Copy Edit

```
<?php
$email = $_POST['email'];
$password = $_POST['password'];

echo "Email: " . htmlspecialchars($email) . "<br>";
echo "Password: " . htmlspecialchars($password);
?>
```

Output is based on submitted input and is not visible in the URL.

Security Note:

Always use functions like `htmlspecialchars()` or `filter_input()` to sanitize inputs and avoid XSS or injection attacks.

Conclusion:

HTML forms are vital for capturing user input in web apps. PHP provides `$_GET` and `$_POST` to handle this data efficiently. The choice depends on the data type, sensitivity, and use case.

2. Explain any Five HTML form input types with usage syntax.

Here are **five commonly used HTML form input types** with their syntax and usage:

1. text

- **Purpose:** Used for single-line plain text input (e.g., name, city).
- **Syntax:**

html

Copy Edit

```
<label for="name">Name:</label>
<input type="text" id="name" name="name">
```

- **Usage:** Useful for general string inputs such as full name, username, etc.
-

2. email

- **Purpose:** Used for email address input. Automatically validates the email format.
- **Syntax:**

```
html Copy Edit  
  
<label for="email">Email:</label>  
<input type="email" id="email" name="email">
```

- **Usage:** Used in registration or contact forms to collect valid email addresses.
-

3. password

- **Purpose:** Used to enter passwords. Characters are hidden for privacy.
- **Syntax:**

```
html Copy Edit  
  
<label for="password">Password:</label>  
<input type="password" id="password" name="password">
```

- **Usage:** Used in login/signup forms to enter confidential passwords.
-

4. radio

- **Purpose:** Used to select one option from a group of choices.
- **Syntax:**

```
html Copy Edit  
  
<label>Gender:</label>  
<input type="radio" name="gender" value="male"> Male  
<input type="radio" name="gender" value="female"> Female
```

- **Usage:** Useful in MCQs, selecting gender, or choosing one option from many.
-

5. checkbox

- **Purpose:** Allows multiple selections from a set of options.
- **Syntax:**

html

CopyEdit

```
<label>Skills:</label><br>
<input type="checkbox" name="skills[]" value="HTML"> HTML
<input type="checkbox" name="skills[]" value="CSS"> CSS
<input type="checkbox" name="skills[]" value="JavaScript"> JavaScript
```

- **Usage:** Used when the user can choose multiple items, like interests or skills.

✔ Summary Table

Input Type	Use Case	Allows Multiple?	Validation
text	Name, city, etc.	No	Manual
email	Email entry	No	Yes
password	Passwords	No	No
radio	One out of multiple	No	No
checkbox	Multiple selections	Yes	No

Let me know if you want examples of how to process these in PHP too!

3. Differentiate between the POST and GET methods in HTML forms. Give code examples to illustrate both methods.

Here is a clear **comparison between the POST and GET methods** in HTML forms, along with examples to illustrate each:

✔ Difference between POST and GET Methods

Feature	GET	POST
Data Visibility	Sent via URL; visible in the address bar	Sent in HTTP body; not visible in URL
Data Length	Limited by URL length (~2048 characters)	No limit (depends on server/browser config)
Use Case	Non-sensitive data, bookmarking/search forms	Sensitive data like login, registration
Caching	Can be cached and bookmarked	Cannot be cached/bookmarked easily
Security	Less secure (exposes data in URL)	More secure for sensitive data
Default Method	Yes, if method is not specified	No

Example 1: Using GET Method

HTML Form (get_form.html):

```
htmlCopy Edit

<form action="get_handler.php" method="get">
  Name: <input type="text" name="name"><br>
  Age: <input type="number" name="age"><br>
  <input type="submit" value="Submit">
</form>
```

PHP Script (get_handler.php):

```
phpCopy Edit

<?php
$name = $_GET['name'];
$age = $_GET['age'];

echo "Name: " . htmlspecialchars($name) . "<br>";
echo "Age: " . htmlspecialchars($age);
?>
```

Example URL after submission:

get_handler.php?name=John&age=25

Example 2: Using POST Method

HTML Form (post_form.html):

html

Copy Edit

```
<form action="post_handler.php" method="post">
  Email: <input type="email" name="email"><br>
  Password: <input type="password" name="password"><br>
  <input type="submit" value="Login">
</form>
```

PHP Script (post_handler.php):

php

Copy Edit

```
<?php
$email = $_POST['email'];
$password = $_POST['password'];

echo "Email: " . htmlspecialchars($email) . "<br>";
echo "Password: " . htmlspecialchars($password);
?>
```

No data shown in the URL — it is securely sent via the request body.

Conclusion

- Use **GET** for data retrieval (search, filtering).
- Use **POST** for actions that **modify or submit sensitive data**.

4. Explain the key terms used in database management systems, highlighting the significance of concepts such as tables, rows, columns, and primary keys

Sure! Below is an explanation of key terms used in **Database Management Systems (DBMS)**, focusing on the essential components such as **tables, rows, columns, and primary keys**:

Key Terms in DBMS

1. Table (Relation)

- A **table** is the core structure of a relational database.
- It represents data in a **structured format** using **rows and columns**.
- Think of a table as a spreadsheet or a grid.

◆ Example:

ID	Name	Email
1	Alice	alice@mail.com
2	Bob	bob@mail.com

- This table might be named users.

2. Row (Record / Tuple)

- A **row** represents a **single record or entry** in a table.
- Each row contains values for all columns in that record.

♦ Example:

(1, "Alice", "alice@mail.com") is one row in the users table.

3. Column (Field / Attribute)

- A **column** defines the **type of data** stored in each field of a record.
- Each column has a **name** and a **data type** (e.g., INT, VARCHAR).

♦ Example Columns in a users table:

- ID – Integer
- Name – String
- Email – String

Each column represents a **particular property** of the data.

4. Primary Key

- A **primary key** is a **unique identifier** for each row in a table.
- It **must contain unique values** and **cannot be NULL**.
- It ensures that no two rows are exactly the same.

♦ Example:

In the users table, ID can be the primary key:

```
sql                                                                    Copy Edit

CREATE TABLE users (
  ID INT PRIMARY KEY,
  Name VARCHAR(100),
  Email VARCHAR(100)
);
```

If ID = 1 is already used, another row cannot have ID = 1.

5. Foreign Key (*Bonus Term*)

- A **foreign key** is a column that creates a **relationship** between two tables.
- It references the **primary key** in another table.

♦ Example:

```
sql                                                                    Copy Edit

CREATE TABLE orders (
  OrderID INT PRIMARY KEY,
  UserID INT,
  FOREIGN KEY (UserID) REFERENCES users(ID)
);
```

UserID in the orders table links to the ID in the users table.

Significance of These Concepts

Term	Importance
Table	Organizes and stores data in structured format
Row	Represents individual data entries
Column	Defines the type of data stored; ensures consistency
Primary Key	Guarantees uniqueness and helps identify rows efficiently
Foreign Key	Maintains referential integrity between tables

5. Explain any five data types in MySQL, including numeric, string, date/time, and Boolean data types, and provide examples of their usage

Here's a detailed explanation of **five commonly used data types in MySQL**, covering **numeric, string, date/time, and Boolean types**, along with usage examples:

✓ 1. INT (Integer)

- **Category:** Numeric
- **Description:** Used to store whole numbers (positive or negative).
- **Storage:** 4 bytes
- **Usage Example:**

```
sql Copy Edit  
  
CREATE TABLE students (  
  id INT PRIMARY KEY,  
  age INT  
);
```

- **Example Value:** 25, -10, 100
-

✓ 2. VARCHAR(n)

- **Category:** String
- **Description:** Variable-length character string; n defines the maximum length.
- **Usage Example:**

```
sql Copy Edit  
  
CREATE TABLE employees (  
  name VARCHAR(100),  
  email VARCHAR(255)  
);
```

- **Example Value:** 'John Doe', 'user@example.com'
-

✓ 3. DATE

- **Category:** Date/Time
- **Description:** Stores date values in YYYY-MM-DD format.
- **Usage Example:**

sql

Copy Edit

```
CREATE TABLE events (  
  event_name VARCHAR(100),  
  event_date DATE  
);
```

- **Example Value:** '2025-06-07'
-

✓ 4. DATETIME

- **Category:** Date/Time
- **Description:** Stores both date and time in YYYY-MM-DD HH:MM:SS format.
- **Usage Example:**

sql

Copy Edit

```
CREATE TABLE logins (  
  user_id INT,  
  login_time DATETIME  
);
```

- **Example Value:** '2025-06-07 12:45:00'
-

✓ 5. BOOLEAN (or TINYINT(1))

- **Category:** Boolean
- **Description:** Represents TRUE or FALSE values. Internally stored as 0 (false) or 1 (true).
- **Usage Example:**

sql

Copy Edit

```
CREATE TABLE users (  
  username VARCHAR(50),  
  is_active BOOLEAN  
);
```

- **Example Value:** 1 (true), 0 (false)
-

 Summary Table

Data Type	Category	Example Value	Description
INT	Numeric	123 , -5	Whole numbers
VARCHAR	String	'Alice' , 'abc123'	Text with variable length
DATE	Date/Time	'2025-06-07'	Stores a date only
DATETIME	Date/Time	'2025-06-07 14:30:00'	Stores both date and time
BOOLEAN	Boolean	1 (true), 0 (false)	Stores true/false status

6. Differentiate between accessing MySQL using the MySQL client and using phpMyAdmin. Explain the advantages and disadvantages of each approach.

Here’s a detailed **comparison** between accessing MySQL via the MySQL client (command-line interface) and phpMyAdmin (web-based interface):

 1. MySQL Client (Command-Line Interface)

◆ **What is it?**

- The **MySQL client** is a **text-based command-line tool** used to interact with MySQL databases.
- Accessed using a terminal or command prompt (e.g., `mysql -u root -p`).

◆ **Advantages:**

Advantage	Description
 Faster for experts	Quick for users familiar with SQL syntax.
 No browser needed	Doesn't rely on a web interface.
 Scripting support	Can write and run SQL scripts using <code>.sql</code> files.
 Remote/SSH access	Can access databases on remote servers via terminal.

◆ **Disadvantages:**

Disadvantage	Description
✗ Steep learning curve	Not user-friendly for beginners.
✗ No graphical interface	Must remember commands and syntax.
✗ Difficult data visualization	Harder to see large tables and relationships clearly.

✓ 2. phpMyAdmin (Web-Based GUI)

◆ What is it?

- **phpMyAdmin** is a **browser-based tool** written in PHP that provides a graphical interface to manage MySQL databases.
- Often installed with XAMPP, WAMP, or LAMP stacks.

◆ Advantages:

Advantage	Description
✓ User-friendly interface	Easy for beginners to learn and use.
✓ Graphical data management	View and manage tables, relationships, and data easily.
✓ No command knowledge needed	Many features like export/import, queries, and backups can be done with clicks.
✓ Visual ER diagrams	View table relations visually.

◆ Disadvantages:

Disadvantage	Description
✗ Slower for large datasets	GUI can lag with huge data tables.
✗ Browser-dependent	Requires a web browser and PHP server.
✗ Security risk if exposed online	Must be properly secured in production.

🔄 Comparison Summary

Feature	MySQL Client	phpMyAdmin
Interface	Text-based (CLI)	Graphical (GUI)
Ease of Use	Harder for beginners	Very easy and intuitive
Speed & Efficiency	Faster for advanced users	Slower for big operations
Installation Required	MySQL only	Requires web server + PHP + MySQL
Scripting Support	Strong (batch files, scripts)	Limited
Visualization	Minimal	Rich (table views, diagrams)

✅ Conclusion:

- Use **MySQL Client** if you're comfortable with SQL, need speed, or are working on remote servers.
- Use **phpMyAdmin** if you're new to MySQL or prefer a graphical interface for managing databases.

7. Outline the essential MySQL commands necessary for database management, including creating databases, tables, inserting data, querying data, and updating records.

Here's a summary of the **essential MySQL commands** for managing databases, covering everything from creating databases to updating records — with syntax and examples:

✅ 1. CREATE DATABASE

Creates a new database.

```
sqlCopyEdit  
  
CREATE DATABASE school;
```

- ◆ **Purpose:** To initialize a new database named school.

✅ 2. USE DATABASE

Selects a specific database to work with.

sql

Copy Edit

```
USE school;
```

- ◆ **Purpose:** Switches to the school database so all further commands apply to it.
-

✓ 3. CREATE TABLE

Creates a new table inside the selected database.

sql

Copy Edit

```
CREATE TABLE students (  
  id INT PRIMARY KEY,  
  name VARCHAR(100),  
  age INT,  
  grade VARCHAR(10)  
);
```

- ◆ **Purpose:** Creates a students table with 4 columns.
-

✓ 4. INSERT INTO

Inserts data into a table.

sql

Copy Edit

```
INSERT INTO students (id, name, age, grade)  
VALUES (1, 'Alice', 20, 'A'),  
      (2, 'Bob', 21, 'B');
```

- ◆ **Purpose:** Adds two student records to the students table.
-

✓ 5. SELECT (Querying Data)

Retrieves data from a table.

a) Select all records:

sql

Copy Edit

```
SELECT * FROM students;
```

b) Select specific columns:

sql

Copy Edit

```
SELECT name, grade FROM students;
```

c) With condition:

sql

Copy Edit

```
SELECT * FROM students WHERE grade = 'A';
```

- ◆ **Purpose:** To view or filter data in the table.

✓ 6. UPDATE

Modifies existing records in a table.

sql

Copy Edit

```
UPDATE students  
SET grade = 'A+'  
WHERE name = 'Alice';
```

- ◆ **Purpose:** Changes Alice's grade to A+.

✓ 7. DELETE

Removes data from a table.

sql

Copy Edit

```
DELETE FROM students WHERE id = 2;
```

- ◆ **Purpose:** Deletes the record with id = 2.

✓ 8. DROP TABLE / DROP DATABASE

Permanently deletes a table or a database.

sql

Copy Edit

```
DROP TABLE students;  
DROP DATABASE school;
```

Purpose: Removes the table or database completely.

✓ 9. ALTER TABLE

Modifies an existing table structure.

```
sql Copy Edit  
  
ALTER TABLE students ADD COLUMN email VARCHAR(100);
```

- ◆ **Purpose:** Adds an email column to the students table.

✦ Summary Table

Task	Command Example
Create Database	CREATE DATABASE school;
Select Database	USE school;
Create Table	CREATE TABLE students (...);
Insert Data	INSERT INTO students (...) VALUES (...);
Query Data	SELECT * FROM students;
Update Data	UPDATE students SET ... WHERE ...;
Delete Data	DELETE FROM students WHERE ...;
Drop Table/Database	DROP TABLE students;
Modify Table Structure	ALTER TABLE students ADD COLUMN ...;

8. Explain the process of connecting to MySQL using PHP . Give the PHP code for connecting and Selecting Specific database.

Here's a clear explanation of how to **connect to MySQL using PHP**, including the **PHP code to establish the connection and select a specific database**:

✓ ◆ Steps to Connect PHP to MySQL:

1. Define connection parameters:

- servername (e.g., localhost)
- username (e.g., root)
- password (e.g., empty or root depending on your setup)

- database name (e.g., school)
2. Use `mysqli_connect()` or `new mysqli()` to connect.
 3. Check for connection errors.

PHP Code to Connect and Select a Database

```
php Copy Edit

<?php
$servername = "localhost";
$username = "root";
$password = ""; // Default password is usually blank for local setups
$dbname = "school"; // The database you want to use

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
    die("✗ Connection failed: " . $conn->connect_error);
}

echo "✅ Connected successfully to database: " . $dbname;
?>
```

✅ Explanation of the Code

Line	Purpose
<code>\$servername</code> , <code>\$username</code> , etc.	Store database connection parameters
<code>new mysqli(...)</code>	Create a new connection object to MySQL
<code>if (\$conn->connect_error)</code>	Check if there was a connection error
<code>die(...)</code>	Stop script execution if connection fails
<code>echo "Connected..."</code>	Display success message if connection is successful

Security Tip

For better security and reusability:

- Move credentials to a separate config.php file.
- Use **environment variables** in production.
- Consider using **PDO** for a more secure, flexible approach (especially for prepared statements).

Let me know if you'd like:

- A version using PDO instead of mysqli
- Or how to fetch data once connected.

9. Explain the process of executing simple queries in MySQL using PHP. Provide a code example demonstrating how to execute a SELECT query and retrieve the results using PHP MySQL functions.

Here's a complete explanation and code example demonstrating how to **execute simple MySQL queries using PHP**, especially a SELECT query to **retrieve data** from a database:

✅ Process of Executing MySQL Queries in PHP

◆ Step-by-step:

1. **Establish a connection** to MySQL using `mysqli_connect()` or `new mysqli()`.
2. **Write a SQL query** (e.g., `SELECT * FROM students`).
3. **Execute the query** using `mysqli_query()`.
4. **Retrieve the results** using `mysqli_fetch_assoc()` or similar.
5. **Display the data** or process it as needed.
6. **Close the connection** with `mysqli_close()`.

📄 PHP Code Example: SELECT Query and Displaying Results

Let's say you have a students table like this:

id	name	age	grade
1	Alice	20	A
2	Bob	21	B

✅ PHP Code:

```
<?php
// Step 1: Connect to MySQL
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "school";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
    die("❌ Connection failed: " . $conn->connect_error);
}

// Step 2: Write the SELECT query
$sql = "SELECT id, name, age, grade FROM students";

// Step 3: Execute the query
$result = $conn->query($sql);

// Step 4: Check and fetch results
if ($result->num_rows > 0) {
    echo "<h3>Student Records:</h3>";
    echo "<ul>";
    while($row = $result->fetch_assoc()) {
        echo "<li>ID: " . $row["id"] . " | Name: " . $row["name"] . " | Age: " . $row["age"] . " | 
Grade: " . $row["grade"] . "</li>";
    }
    echo "</ul>";
} else {
    echo "No records found.";
}

// Step 5: Close connection
$conn->close();
?>
```

✅ What's Happening Here?

Part	Explanation
<code>\$conn = new mysqli(...)</code>	Connects to the MySQL database
<code>\$sql = "SELECT..."</code>	Defines the SQL query
<code>\$conn->query(\$sql)</code>	Executes the SQL and returns a result set
<code>while(\$row = ...)</code>	Iterates through each row and displays the data
<code>\$conn->close()</code>	Closes the database connection

✂ Output Example:

```
yaml                                                                    Copy Edit
Student Records:
- ID: 1 | Name: Alice | Age: 20 | Grade: A
- ID: 2 | Name: Bob | Age: 21 | Grade: B
```

10. Explain the process of updating records in a MySQL database.

✅ 10. Explaining the Process of Updating Records in a MySQL Database

Updating records in a MySQL database means **modifying existing data** in a table using the UPDATE SQL statement.

◆ Basic Syntax of UPDATE

```
sql                                                                    Copy Edit
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

- `table_name`: Name of the table where you want to update the record.
- `SET`: Specifies the column(s) and the new value(s).
- `WHERE`: Specifies which row(s) should be updated (important to avoid updating all rows).

✅ Example in MySQL (SQL only)

Suppose you have a students table:

id	name	age	grade
1	Alice	20	B
2	Bob	21	C

▶ SQL Query to update Alice's grade:

sql

Copy Edit

```
UPDATE students
SET grade = 'A'
WHERE name = 'Alice';
```

This will change Alice's grade from 'B' to 'A'.

📄 PHP Code Example: Updating a Record

▶ PHP Script:

php

Copy Edit

```
<?php
// Step 1: Connect to MySQL
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "school";

$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
    die("❌ Connection failed: " . $conn->connect_error);
}

// Step 2: Write the UPDATE query
$sql = "UPDATE students SET grade = 'A' WHERE name = 'Alice'";

// Step 3: Execute the query
if ($conn->query($sql) === TRUE) {
    echo "✅ Record updated successfully!";
}
```

```

    } else {
        echo "❌ Error updating record: " . $conn->error;
    }

    // Step 4: Close the connection
    $conn->close();
?>

```

🔒 Important Tips:

Tip	Why it matters
Always use <code>WHERE</code> clause	To avoid updating all rows by mistake.
Test query with <code>SELECT</code> before running <code>UPDATE</code>	To preview affected rows.
Use <code>LIMIT</code> clause (optional)	Restrict updates to a fixed number of rows.

Example:

sql	📋 Copy	✎ Edit
<pre>UPDATE students SET grade = 'B' WHERE grade = 'C' LIMIT 5;</pre>		

✅ Summary:

- UPDATE is used to **modify existing data**.
- Always pair it with a WHERE clause to target specific rows.
- PHP + MySQL can execute UPDATE queries using `$conn->query($sql)`.

11. Explain the process of retrieving query results in PHP after executing a SELECT statement in MySQL with code example.

✅ 1. Process of Retrieving Query Results in PHP After Executing a SELECT Statement in MySQL

Retrieving data from a MySQL database using PHP involves the following steps:

◆ Step-by-Step Process

1. **Connect to the MySQL database** using `mysqli` or `PDO`.

2. **Write a SELECT query.**
 3. **Execute the query** using `mysqli_query()` or `$conn->query()`.
 4. **Fetch the results** using functions like:
 - `mysqli_fetch_assoc()` — returns associative array
 - `mysqli_fetch_row()` — returns indexed array
 - `mysqli_fetch_array()` — returns both
 5. **Loop through the data** to display it.
 6. **Close the connection.**
-

PHP Code Example: Using mysqli

Assume you have a table `students(id, name, age, grade)`.

Full Code Example:

php

 Copy  Edit

```
<?php
// 1. Connect to the database
$servername = "localhost";
$username = "root";
$password = "";
$database = "school";

$conn = new mysqli($servername, $username, $password, $database);

// Check connection
if ($conn->connect_error) {
    die("✗ Connection failed: " . $conn->connect_error);
}

// 2. Write SELECT query
$sql = "SELECT id, name, age, grade FROM students";

// 3. Execute the query
$result = $conn->query($sql);

// 4. Check and fetch results
if ($result->num_rows > 0) {
    echo "<h3>Student Records:</h3>";
    echo "<table border='1'><tr><th>ID</th><th>Name</th><th>Age</th><th>Grade</th></tr>";
```

```
// 5. Loop through each row
while($row = $result->fetch_assoc()) {
    echo "<tr>
        <td>" . $row["id"] . "</td>
        <td>" . $row["name"] . "</td>
        <td>" . $row["age"] . "</td>
        <td>" . $row["grade"] . "</td>
    </tr>";
}
echo "</table>";
} else {
    echo "No records found.";
}

// 6. Close the connection
$conn->close();
?>
```

✓ Output Example (Browser)

markdown

Copy Edit

Student Records:

```
-----
| ID | Name | Age | Grade |
|----|-----|----|-----|
| 1  | Alice | 20  | A     |
| 2  | Bob   | 21  | B     |
```

🔄 Alternative Fetching Functions

Function	Description
<code>mysqli_fetch_assoc()</code>	Returns row as associative array
<code>mysqli_fetch_row()</code>	Returns row as indexed array
<code>mysqli_fetch_array()</code>	Returns both associative and indexed

🔒 Note:

- You should **sanitize user input** and use **prepared statements** if you're fetching data based on user input to prevent SQL injection.